KITCHEN MATE

RECIPE RECOMMENDATION SYSTEM

USING PYTHON AND TKINTER

Submitted by

Parv Attrey - 26MCA20069

in partial fulfillment for the award of the degree of

Master of Computer Applications (MCA)

IN

Computer Applications



Chandigarh University

October - 2026

1. Project Title and Introduction

Title: Kitchen Mate - Your recipe recommendation system

Introduction:

Kitchen Mate is an intuitive, Python-based desktop application designed to solve the everyday dilemma of "what to cook" with currently available ingredients. The primary focus of the application is to reduce food waste and simplify meal planning by encouraging users to utilize their existing pantry items. By integrating a built-in database of 50 traditional Indian vegetarian recipes, the platform algorithmically curates personalized recipe recommendations based on user input. Presented through a modern graphical user interface (GUI) built with Tkinter, the application makes home cooking engaging, accessible, and highly practical.

2. Problem Statement and Objectives

Problem Statement:

In daily household environments, individuals frequently struggle with meal planning based on leftover or random pantry ingredients. This lack of culinary inspiration often leads to food waste, unnecessary grocery expenses, and the reliance on fast food. Users lack an immediate, offline resource to intelligently match the ingredients they already possess with actionable, step-by-step recipes.

Objectives:

Develop an accessible, offline desktop application that promotes resourceful cooking and reduces household food waste.

Implement a responsive search algorithm to provide immediate, tailored recipe solutions based on comma-separated ingredient lists.

Deliver a clean, user-friendly interface utilizing modern UI components (Themed Tkinter)

Facilitate the growth of a personalized digital cookbook by allowing users to add, save, and persistently store their own recipes.

3. Tools and Technologies Used

Programming Logic & Core Language: Python 3 (Vanilla). Handles all data processing, string manipulation, and logical routing.

Graphical User Interface (GUI): Tkinter and tkinter.ttk (Themed Tkinter). Utilized to implement a highly responsive, modern interface featuring tabbed navigation (Notebook), split panes (PanedWindow), and scrollable text areas.

Data Storage & Management: Python CSV Module. Functions as a lightweight, persistent local database (kitchen_mate_recipes.csv) to read pre-loaded recipes and append new user-generated recipes without requiring a heavy SQL server.

File System Integration: Python OS Module. Ensures smooth initialization by checking for existing database files and automatically generating the default 50 Indian Veg recipes upon the first run.

4. Algorithm / Methodology

The application follows an Object-Oriented Programming (OOP) architecture, emphasizing dynamic data loading and efficient string matching.

Ingredient Matching Algorithm: The core "Smart Chef" feature operates on a substring checking methodology. When a user inputs their inventory (e.g., "tomato, paneer"), the system sanitizes the input by converting it to lowercase and stripping whitespace. It then iterates through the recipe database, checking if any of the user's inventory items exist as a substring within the required ingredients of each recipe.

Scoring and Sorting Mechanism: To ensure the best recommendations appear first, the algorithm calculates a match score for every recipe using the formula: (Matched Ingredients / Total Required Ingredients) * 100. The resulting list is sorted in descending order based on this score, displaying the most viable recipes at the top of the list.

Persistent Data Handling: The application employs a robust I/O methodology. On startup, it verifies the existence of the CSV file. If absent, it writes the default dataset to disk. All user additions are appended directly to this CSV, and the application state is immediately refreshed to reflect the new data in both the "Smart Chef" and "Recipe Book" tabs.

Here is the step-by-step algorithm for the core ingredient matching and data handling process used in the project:

Step 1: Initialization and Data Loading

- Start the application and verify if the local database file (kitchen_mate_recipes.csv) exists.
- If the file does not exist, automatically generate it and populate it with 50 default Indian vegetarian recipes.
- Open the CSV file, read all rows, and load the recipe data into a Python list of dictionaries (memory) for fast access.

Step 2: User Input Processing

- Capture the comma-separated string of ingredients entered by the user in the "Smart Chef" tab.
- Validate the input: if the field is empty or contains the default placeholder text, display an error warning and halt the process.
- Sanitize the input: split the string by commas, remove leading/trailing whitespace from each item, and convert all text to lowercase to create the inventory list.

Step 3: Recipe Iteration and Sanitization

- Initialize an empty list called matches to store successful recipe matches.
- Begin a loop to iterate through every recipe stored in the loaded database.
- For the current recipe, extract its required ingredients, split them by commas, strip whitespace, and convert them to lowercase.

Step 4: Substring Matching Logic

- Initialize a matched_count variable to 0 for the current recipe.
- Iterate through each required ingredient of the current recipe.
- For each required ingredient, iterate through the user's sanitized inventory list.

- Check if the user's inventory item is a substring of the required ingredient (e.g., checking if "tomato" is in "red tomato").
- If a match is found, increment the matched_count by 1, stop checking the current required ingredient, and move to the next one.

Step 5: Scoring

- After checking all required ingredients for the current recipe, determine the total_req (total number of ingredients required for that dish).
- If matched_count is greater than 0, calculate the match percentage score using the formula: (matched_count / total_req) * 100.
- Append a dictionary containing the recipe details, the calculated score, the matched_count, and the total_req to the matches list.

Step 6: Sorting Results

- Once all recipes have been evaluated, sort the matches list in descending order based on the calculated percentage score, ensuring the most relevant recipes appear first.

Step 7: User Interface (UI) Update

- Clear any existing data from the results Listbox in the GUI.
- If the matches list is empty, display a "No matching recipes found" message.
- If matches exist, loop through the sorted matches list and insert each recipe's name alongside its match fraction (e.g., "Aloo Gobi (2/7 match)") into the Listbox for the user to select.

5. Implementation

The implementation is segmented into three primary functional tabs within a single main application window:

Smart Chef (Tab 1): The core interactive component. Features a text entry field for inventory and a split-pane layout. The left pane contains a clickable Listbox displaying sorted match results, while the right pane dynamically renders the formatted recipe details (Ingredients and Instructions) upon selection.

Recipe Book (Tab 2): A browsing interface. It loads the entire recipe database, sorts it alphabetically, and displays it in a Listbox for easy navigation and reading.

- Add Recipe (Tab 3): A data-entry form consisting of labeled Entry and Text widgets. It captures the Recipe Name, Ingredients, and Instructions, validates that no fields are empty, and appends the data to the CSV file with error handling.

5.1 Core Code Snippet (Matching Algorithm)

```
matches = []
for recipe in self.recipes:
    recipe_ingredients = [ing.strip().lower() for ing in recipe['Ingredients'].split(",")]
    matched_count = 0

    for req_ing in recipe_ingredients:
        for inv_item in inventory:
            if inv_item in req_ing: # Substring match
                matched_count += 1
                break

    total_req = len(recipe_ingredients)

    if matched_count > 0:
        score = (matched_count / total_req) * 100
        matches.append({"recipe": recipe, "score": score, "matched": matched_count, "total": total_req})

matches.sort(key=lambda x: x['score'], reverse=True)
```

6. Output / Results (Screenshots)

Figure 1:

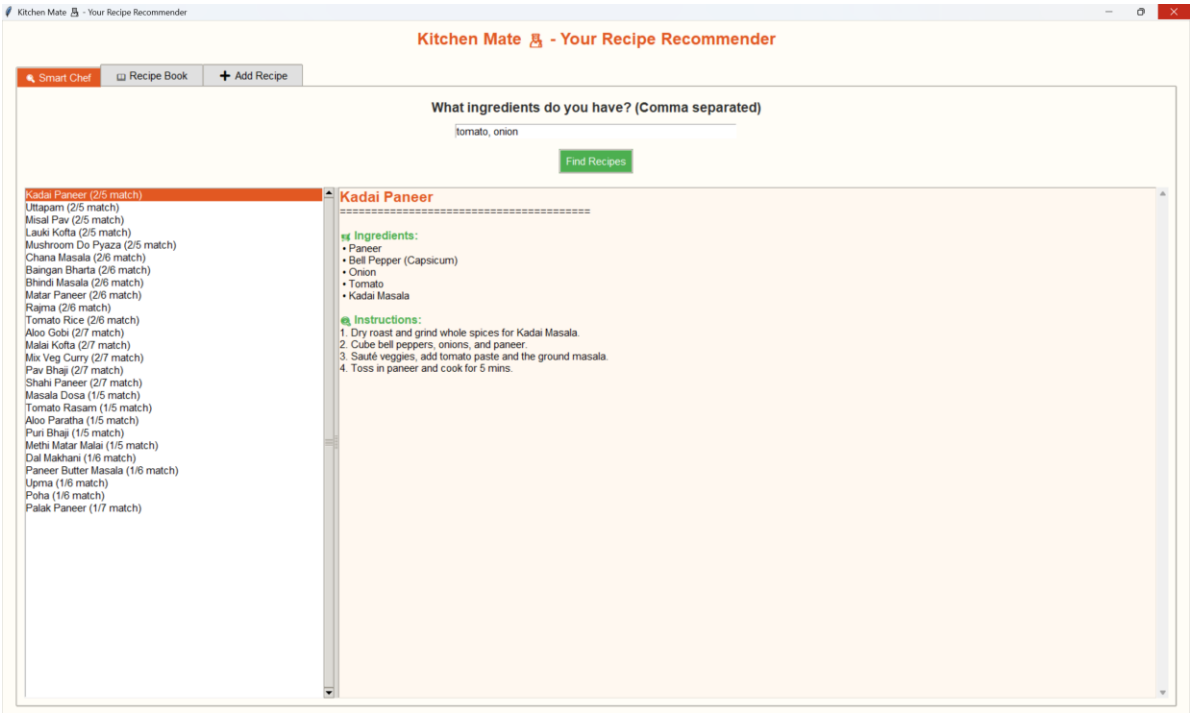


Figure 2:

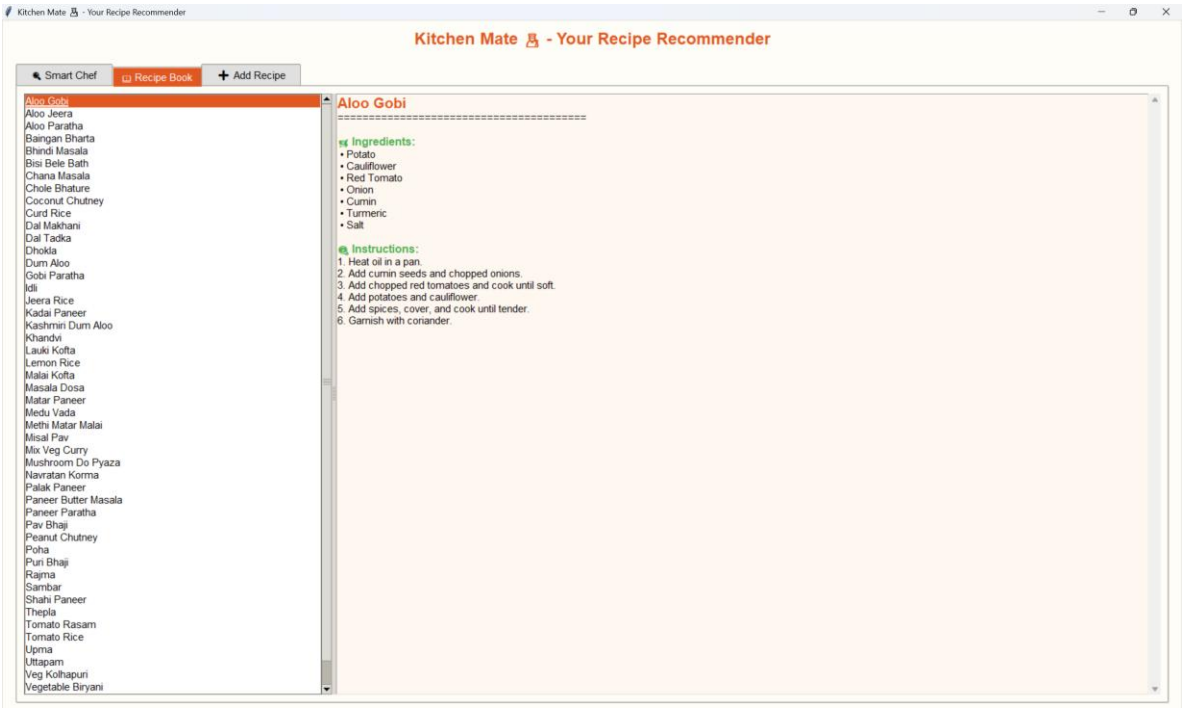
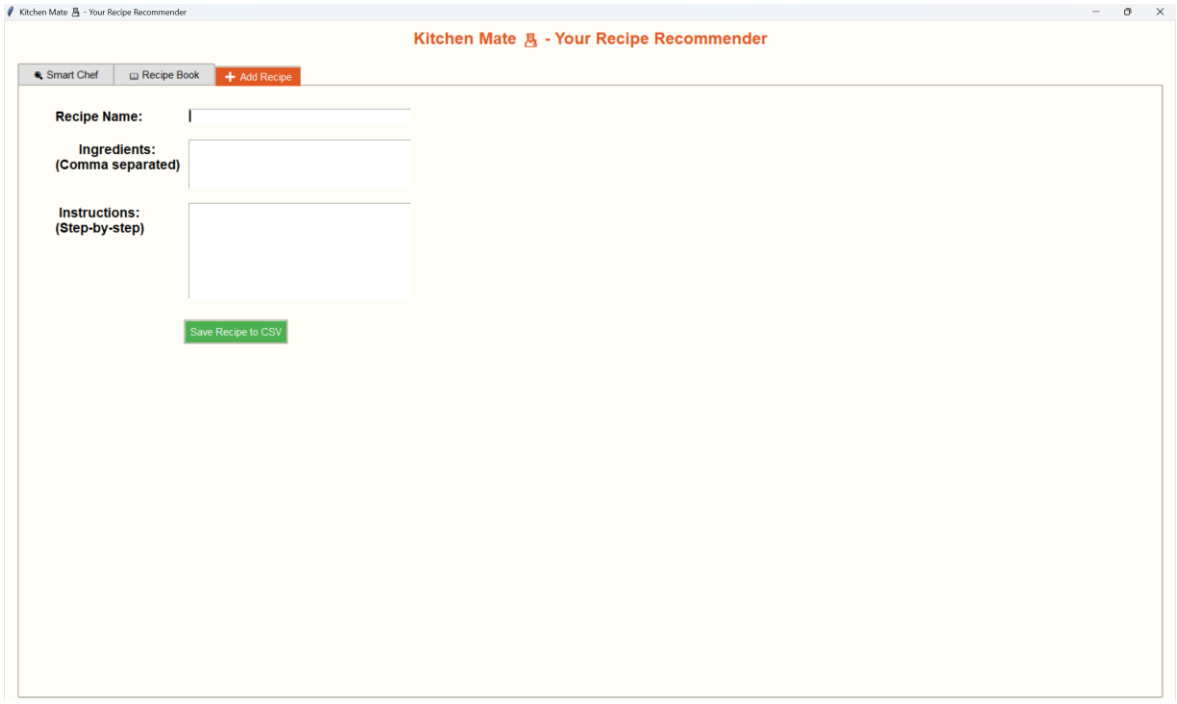


Figure 3:



7. Conclusion

The Kitchen Mate project successfully demonstrates the utility of Python and the Tkinter framework in creating practical, locally hosted desktop applications. By implementing a responsive substring-matching algorithm and persistent CSV storage, the application effectively solves the problem of daily meal planning and food resource management. The modular OOP design and intuitive tabbed interface make it a highly functional proof-of-concept for how lightweight software tools can promote self-sufficiency and reduce household waste.